第31课 栈与队列（STL）

一、课程信息

项目	内容
课次	第 31 课 / 共 40 课
主题	栈与队列（STL）
前置知识	数组、循环、函数、递归
后续衔接	第 32 课——vector 与常用 STL

教学目标（可检验）

1. **能用 `stack` 完成基本操作**：学生能独立写出使用 `push()`、`pop()`、`top()`、`empty()`、`size()` 的完整程序，并正确运行。
2. **能用 `queue` 完成基本操作**：学生能独立写出使用 `push()`、`pop()`、`front()`、`back()`、`empty()`、`size()` 的完整程序，并正确运行。
3. **能用栈或队列解决简单应用题**：学生能用栈完成括号匹配判断，能用队列模拟排队过程。

二、教学重点与难点

重点

- 栈"后进先出"（LIFO）与队列"先进先出"（FIFO）的核心概念区分。
- `stack` 和 `queue` 的头文件引入与五/六个基本操作函数的使用。
- 在操作前判断是否为空（`empty()`），避免对空栈/空队列调用 `top()` / `front()` / `pop()`。

难点

- **栈与队列的"取"和"删"是分开的**：`top()` / `front()` 只看不删，`pop()` 只删不看，需要组合使用。初学者容易混淆。
- **括号匹配的逻辑**：遇到左括号入栈、遇到右括号出栈检查——需要将生活直觉转化为代码逻辑。
- **区分栈和队列的 `pop()` 删除位置不同**：栈删栈顶（最后进的），队列删队首（最先进的）。

三、课堂时间分配（共 100 分钟）

时间	环节	内容	形式
0′–5′	导入	生活故事引入：一摞盘子 vs 排队买饭	讲述+互动提问
5′–20′	讲解一	栈的概念 + `stack` 基本操作演示	讲解+板书（≤15 min）
20′–35′	练习一	例题 1 跟敲 + 课堂练习 1	学生动手
35′–50′	讲解二	队列的概念 + `queue` 基本操作演示	讲解+板书（≤15 min）
50′–65′	练习二	例题 2 跟敲 + 课堂练习 2	学生动手
65′–80′	讲解三	栈的应用——括号匹配（例题 3）	讲解+边写边练（≤15 min）
80′–95′	练习三	队列应用——模拟排队（例题 4）+ 课堂练习 3-4	学生动手
95′–100′	总结	栈 vs 队列对比表 + 作业布置 + 下节预告	讲述

> **设计原则**：任何一段连续讲解不超过 25 分钟，讲练穿插，保持注意力。

四、讲稿提纲

4.1 导入故事（0′–5′）

> 同学们好！今天我们来认识两个新朋友——**栈**和**队列**。
>
> 先讲个小故事：
>
> **场景一**：食堂阿姨洗好了盘子，一个一个往上摞。你去拿盘子的时候，是不是只能拿最上面那个？最后放上去的盘子，反而最先被拿走——这就叫**"后进先出"**，我们给它起个名字叫**栈（zhàn）**。你也可以想象成手枪的弹夹：最后压进去的子弹，最先射出来。
>
> **场景二**：中午去食堂排队打饭，谁先排的队？当然是队伍最前面的人先打到饭。先来的先走——这就叫**"先进先出"**，我们给它起个名字叫**队列**。
>
> 今天我们就来学习怎么用 C++ 来"摞盘子"和"排队"！

4.2 讲解一：栈 `stack`（5′–20′）

要点提纲：

1. **什么是栈?**
   - 画图演示：竖着的一摞积木，只能从最顶上放、也只能从最顶上拿。
   - 关键词：**后进先出（LIFO — Last In, First Out）**。

2. **C++ 怎么用栈?**
   - 头文件：`#include <stack>`
   - 声明：`stack<int> s;`（尖括号里写放什么类型的数据）
   - 类比：`stack<int>` 就像"一摞写着数字的盘子"。
3. **五个基本操作**（边讲边在黑板画盘子动画）：

操作	含义	生活比喻
`s.push(x)`	把 x 放到栈顶	把一个盘子摞上去
`s.top()`	看一眼栈顶是什么（不拿走）	看看最上面的盘子是哪个
`s.pop()`	把栈顶拿走（不告诉你是什么）	把最上面的盘子端走
`s.empty()`	栈里还有没有东西?	盘子摞完了吗?
`s.size()`	栈里有几个元素	数一数还剩几个盘子

1. **重要提醒**（红笔标注）：
   - `top()` 只是"看"，`pop()` 只是"删"，想先看再删要写两行!
   - 对空栈调用 `top()` 或 `pop()` 会出错，操作前先用 `empty()` 检查!

4.3 练习一：例题 1 跟敲 + 课堂练习 1（20′–35′）

> 详见 **第五章 例题 1** 和 **第六章 练习 1**。

4.4 讲解二：队列 `queue`（35′–50′）

要点提纲:

1. **什么是队列?**
   - 画图演示：一排小朋友排队，从队尾进、从队首出。
   - 关键词：**先进先出（FIFO — First In, First Out）**。
2. **C++ 怎么用队列?**
   - 头文件：`#include <queue>`
   - 声明：`queue<int> q;`
   - 类比：`queue<int>` 就像"排队买冰淇淋的队伍，每个人身上贴了一个数字"。
3. **六个基本操作**（边讲边画队伍动画）：

操作	含义	生活比喻
`q.push(x)`	从队尾加入 x	新同学排到队尾
`q.front()`	看一眼队首是谁（不走）	看看谁排在最前面
`q.back()`	看一眼队尾是谁	看看谁排在最后面
`q.pop()`	队首那个人离开	最前面的同学打完饭走了
`q.empty()`	队伍里还有人吗?	队伍排空了吗?
`q.size()`	队伍里有几个人	数一数队伍还有多长

1. **栈 vs 队列 对比**：
   - 栈—— `top()` 看栈顶；队列—— `front()` 看队首， `back()` 看队尾。
   - 栈——从顶部进出；队列——从尾部进、从头部出。
   - 都用 `push()` 添加、 `pop()` 删除，但删除的位置不同！

4.5 练习二：例题 2 跟敲 + 课堂练习 2（50′–65′）

> 详见 **第五章 例题 2** 和 **第六章 练习 2**。

4.6 讲解三：栈的应用——括号匹配（65′–80′）

要点提纲：

1. **生活引入**：

   > 写数学算式的时候，括号要成对出现： `(1 + 2) * 3` 是对的， `(1 + 2 * 3` 就少了右括号。
   > 怎么让计算机帮我们检查括号对不对呢？用栈！

2. **思路讲解**（配合动画演示）：
   - 从左到右一个一个看字符。
   - 遇到 `(` ，压入栈——相当于"记住这里有个等待配对的左括号"。
   - 遇到 `)`，检查栈顶有没有 `(`：
   - 有 配对成功，弹出栈顶。
   - 没有（栈为空） 多了右括号，不匹配！
   - 全部看完后，如果栈为空 全部配对成功；栈不为空 有多余的左括号。

3. **边写代码边讲**：见例题 3。

4.7 练习三：队列应用 + 综合练习（80′–95′）

> 详见 **第五章 例题 4** 和 **第六章 练习 3-4**。

4.8 总结与预告（95′–100′）

> **快速对比**（投屏或板书）：
>
> | | 栈 Stack | 队列 Queue |
> |---|---|---|
> | 原则 | 后进先出 LIFO | 先进先出 FIFO |
> | 头文件 | `<stack>` | `<queue>` |
> | 添加 | `push()` | `push()` |
> | 查看 | `top()` | `front()` / `back()` |
> | 删除 | `pop()` | `pop()` |
> | 判空 | `empty()` | `empty()` |
> | 大小 | `size()` | `size()` |
>
> 大家今天学了两个非常实用的"容器"，它们在以后的搜索算法（DFS 用栈、BFS 用队列）中会大量出现，是非常重要的基础！

七、常见错误与应对

错误 1：对空栈/空队列调用 `top()`、`front()` 或 `pop()`

```
stack<int> s;
cout << s.top();  //  运行时错误！栈是空的，没有栈顶
```

应对：养成好习惯——操作前先检查 `empty()`。

```
if (!s.empty()) {
    cout << s.top();
}
```

> **老师说**：就像你不能从空盘子堆上拿盘子一样，先看看有没有盘子！

错误 2：以为 `pop()` 会返回被删除的值

```
stack<int> s;
s.push(10);
int x = s.pop();  //  编译错误! pop() 返回 void，不返回值
```

应对：先用 `top()` 取值，再 `pop()` 删除。

```
int x = s.top();  // 先看
s.pop();           // 再删
```

> **老师说**：`top()` 是"看一眼"，`pop()` 是"扔掉"。想知道扔掉的是什么，得先看一眼记住，再扔！

错误 3：混淆栈的 `top()` 和队列的 `front()`

```
queue<int> q;
q.push(1);
cout << q.top();  //  编译错误! 队列没有 top()，要用 front()
```

应对：记忆口诀——**"栈看顶 top，队看头 front"**。

错误 4：忘记写头文件

```
// 忘记 #include <stack>
stack<int> s;  //  编译错误
```

应对：用栈就 `#include <stack>`，用队列就 `#include <queue>`。建议写完 `#include` 后立刻编译一次确认。

错误 5：括号匹配时只检查了栈空，忘记检查多余左括号

```
// 只在循环里检查了"右括号多了"的情况
// 循环结束后忘记判断栈是否为空
```

应对：循环结束后一定要加 `if (!s.empty()) ok = false;`，否则 `(((` 这样的输入会被误判为正确。

九、下节课预告

> **第 32 课：vector 与常用 STL**
>
> 今天我们认识了栈和队列这两个 STL 容器。下节课，我们要认识一个更强大、更灵活的容器——`vector`**（动态数组）**！它就像一个"会自动变长的数组"，想放多少数字就放多少，再也不用提前定义大小了。我们还会认识几个常用的 STL 工具，让写代码变得更轻松！
>
> 预习小任务：想一想，普通数组有什么不方便的地方？如果数组能"自动变长"会有什么好处？